



**THE YENEPOYA INSTITUTE OF ARTS SCIENCE
COMMERCE AND MANAGEMENT
A CONSTITUENT UNIT OF YENEPOYA DEEMED TO BE
UNIVERSITY
BALMATTA, MANGALORE**

LOW LEVEL DESIGN ON

**Natural Language Interface for Agriculture
Infrastructure**

**BCA AI, CLOUD COMPUTING & DEVOPS WITH IBM
COMPUTER SCIENCE**

**SUBMITTED BY:
MOHAMMED SHAN S
23BCAICD066
25755
25755@yenepoya.edu.in**

**Industry Mentor: Sumit Kumar Shukla
Guided by: Deepthi**

Table of Contents

1. Introduction.....	3
1.1 Purpose.....	3
1.2 Scope.....	3
1.3 Definitions	3
2. System Design.....	3
2.1 Architecture Overview.....	3
2.2 Module Overview	4
3. Detailed Module Design.....	5
3.1 Configuration & Session Module.....	5
3.2 Utility Module.....	5
3.3 Knowledge Base Module.....	5
3.4 Query Processor Module.....	5
3.5 UI & Control Flow Module	6
4. Function-Level Design	6
5. Data Structure Design	6
6. UI Design	7
7. Algorithm Design	7
8. Error Handling	7
9. Testing	8
10. Deployment	8
11. Future Enhancements	8
12. Conclusion	8
13. References	9

1. Introduction

1.1 Purpose

This document provides the Low-Level Design of an AI-powered Agriculture Assistant chatbot. The system is designed to assist users in obtaining crop information, fertilizer recommendations, weather updates, and market prices through an interactive chat interface.

1.2 Scope

The design includes internal modules, processing logic, and data structures. The system currently uses structured data and simulated responses but is designed to support integration with real-time APIs and AI models.

1.3 Definitions

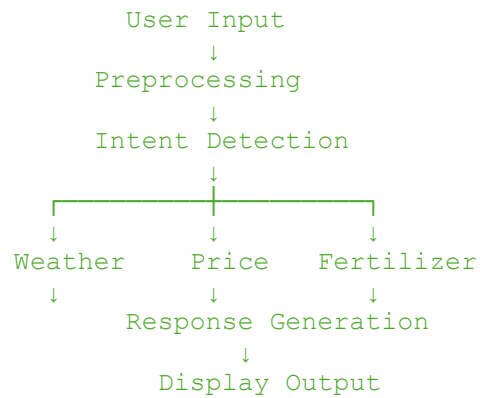
Intent represents the purpose behind a user query. A session refers to the continuous interaction between the user and the system. The knowledge base stores crop-related data, and NPK refers to essential nutrients required for plant growth.

2. System Design

2.1 Architecture Overview

The system follows a simple layered architecture. The user interacts with the chat interface, which sends input to the backend logic. The input is preprocessed and analyzed to detect intent. Based on the detected intent, the system routes the request to the appropriate module such as weather, market price, fertilizer, or crop advice. The response is then generated and displayed to the user.

System Flowchart



2.2 Module Overview

Module	Responsibility
Configuration	Initialize application and styling
Session	Maintain chat history and state
Utility	Provide helper functions
Knowledge Base	Store crop information
Query Processor	Detect intent and route queries
UI	Display chat interface
Control Flow	Manage execution flow

3. Detailed Module Design

3.1 Configuration & Session Module

This module initializes the application and manages session data such as chat history, session ID, and message count. It ensures continuity of interaction and maintains system state during runtime.

3.2 Utility Module

The utility module provides helper functions such as text preprocessing, time detection, and random value generation. These functions support the main processing logic and improve code modularity.

3.3 Knowledge Base Module

The knowledge base stores crop-related data including nutrient values, growing conditions, and duration. It serves as the primary source of information for generating crop advice and fertilizer recommendations.

3.4 Query Processor Module

This is the core module of the system. It processes user input, identifies intent using keyword matching or basic NLP techniques, and routes the request to the appropriate function. It ensures correct and efficient response generation.

3.5 UI & Control Flow Module

This module manages user interaction and system execution. It displays chat messages, handles user input, and controls the sequence of operations within the application.

4. Function-Level Design

The system includes key functions responsible for handling user queries. The main function processes the input, detects intent, and generates responses accordingly.

The intent classification function determines the category of the query such as weather, price, fertilizer, or crop advice. Based on this classification, the system calls the relevant function.

The fertilizer calculation function computes nutrient requirements based on crop data and land area. Weather and market price functions generate responses using either simulated data or structured datasets.

5. Data Structure Design

The system uses structured data to manage information efficiently. The knowledge base stores crop-related details such as nutrient values, season, and soil type.

Chat messages are stored as structured objects containing the role and content of each message. Market data includes minimum, maximum, and average prices along with trends. Session data maintains chat history and interaction details during execution.

6. UI Design

The user interface is built using Streamlit and follows a chat-based layout. Messages are displayed in a conversational format with clear separation between user input and system responses.

The interface also includes quick action buttons for commonly used features such as weather updates, crop information, and fertilizer calculation. This improves usability and reduces user effort.

7. Algorithm Design

The system uses a simple intent detection approach based on keyword matching or basic NLP logic. Once the intent is identified, the query is routed to the corresponding function.

Fertilizer calculation is performed using predefined formulas based on nutrient requirements. Weather and market price information are generated through simulation or structured data retrieval.

8. Error Handling

The system handles common errors such as empty input, unknown crop names, and missing parameters. It provides user-friendly prompts to guide the user in correcting their input. A general error handler ensures system stability and prevents crashes.

9. Testing

Testing is performed to ensure correctness and reliability. Individual components are tested for functionality, and complete system testing verifies the interaction flow. Edge cases such as invalid input and unknown queries are also tested.

10. Deployment

The system is deployed using Streamlit and can run locally or on cloud platforms. It requires Python and necessary libraries and can be executed using a simple command. The lightweight design ensures easy deployment.

11. Future Enhancements

Future improvements include integrating advanced AI models, connecting to real-time APIs, and adding features such as voice interaction and multi-language support to enhance usability.

12. Conclusion

The AI-powered Agriculture Assistant provides an efficient and user-friendly solution for handling agricultural queries. Its modular design ensures scalability, maintainability, and ease of future enhancements.

13. References

- Python Documentation: <https://docs.python.org/3/>
- Streamlit Documentation: <https://docs.streamlit.io/>
- OpenWeather API: <https://openweathermap.org/>
- FAO: <https://www.fao.org/>
- ICAR: <https://icar.org.in/>